

Notas sobre Métodos Numéricos con R

Carlos E. Martínez-Rodríguez
Universidad Autónoma de la Ciudad de México
Academia de Matemáticas
`carlos.martinez@uacm.edu.mx`

Índice

1. Interpolación Numérica	2
1.1. Conceptos Fundamentales	2
1.2. Forma General del Polinomio Interpolante	3
1.3. Interpolación de Lagrange	5
1.4. Interpolación de Newton–Diferencias Finitas	6
1.5. Splines	18
1.6. Error de Interpolación	21
1.7. Resumen	22
1.8. Ejercicios	23
1.9. Implementaciones Computacionales	30

1. Interpolación Numérica

1.1. Conceptos Fundamentales

Definición 1. *La interpolación es una técnica numérica utilizada para estimar valores desconocidos de una función a partir de un conjunto de puntos conocidos. A diferencia del ajuste de curvas, la interpolación busca una función que pase exactamente por los puntos dados. La interpolación consiste en estimar el valor de una función dentro del intervalo cubierto por un conjunto de datos conocidos.*

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n) \quad (1)$$

donde se asume que $y_i = f(x_i)$ para $i = 0, 1, \dots, n$. El objetivo es construir una función $P(x)$, generalmente un polinomio, tal que:

$$P(x_i) = f(x_i) = y_i, \quad i = 0, 1, \dots, n. \quad (2)$$

Dado que existen $n + 1$ puntos distintos (x_i, y_i) , se puede demostrar que existe un único polinomio de grado n que los interpola:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

La determinación de los coeficientes a_i puede hacerse de diversas maneras, dando origen a los distintos métodos de interpolación.

Nota 1. En la interpolación, la función construida pasa exactamente por los puntos dados; en el ajuste de curvas, se busca una función que aproxime los datos minimizando un error global. Si el polinomio interpolante se usa para estimar valores de $f(x)$ fuera del intervalo cubierto por los puntos dados, el proceso se denomina **extrapolación**. La extrapolación extiende esta idea a valores fuera del intervalo, con menor confiabilidad, tiende a generar errores grandes, por lo que debe usarse con precaución.

Comparación entre interpolación y ajuste de curvas:

Interpolación	Ajuste de Curvas
La curva pasa exactamente por todos los puntos.	La curva se aproxima a los puntos, pero no necesariamente pasa por todos.
Se usa cuando los datos son exactos o tabulados sin error experimental.	Se usa cuando los datos contienen ruido o errores de medición.
Ejemplo: tablas de logaritmos o funciones trigonométricas.	Ejemplo: datos experimentales en laboratorio.

Nota 2. El error de interpolación mide la diferencia entre el valor real de la función y el valor estimado por el polinomio interpolante. Se expresa mediante la fórmula:

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i), \quad (3)$$

para algún ξ en el intervalo de interpolación.

El error cometido al aproximar $f(x)$ mediante el polinomio interpolante $P_n(x)$ se expresa como:

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

donde ξ es algún valor dentro del intervalo (x_0, x_n) . El error depende de la derivada de orden $(n+1)$ de la función $f(x)$ y de la distribución de los nodos x_i .

Nota 3. Problemas mal condicionados (efecto Runge): El uso de polinomios de alto grado con nodos equiespaciados puede producir oscilaciones extremas cerca de los extremos del intervalo. Cuando los nodos x_i están equiespaciados y se incrementa el número de puntos, el polinomio interpolante puede oscilar significativamente, especialmente cerca de los extremos del intervalo. Este fenómeno se conoce como **efecto Runge**. Una forma de mitigarlo es usar nodos distribuidos según los **polinomios de Chebyshev** o recurrir a métodos por tramos como los

1.2. Forma General del Polinomio Interpolante

Se define el polinomio de grado n que pasa por $n+1$ puntos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$.

$$P_n(x_i) = y_i, \quad i = 0, 1, 2, \dots, n$$

donde se conoce el valor de la función $f(x)$ en cada punto, es decir, $y_i = f(x_i)$. El **problema de interpolación polinómica** consiste en determinar un polinomio $P_n(x)$ de grado menor o igual a n , la forma general del polinomio interpolante es:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n,$$

donde los coeficientes $a_0, a_1, \dots, a_n$ se determinan imponiendo las condiciones de interpolación:

$$\begin{cases} a_0 + a_1x_0 + a_2x_0^2 + \dots + a_nx_0^n = y_0 \\ a_0 + a_1x_1 + a_2x_1^2 + \dots + a_nx_1^n = y_1 \\ \vdots \\ a_0 + a_1x_n + a_2x_n^2 + \dots + a_nx_n^n = y_n \end{cases}$$

En forma matricial:

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix}}_V \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}}_{\vec{a}} = \underbrace{\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}}_{\vec{y}}$$

donde V es la **matriz de Vandermonde**. La existencia de una solución única depende de que V sea invertible, lo cual se cumple siempre que todos los x_i sean distintos. Para demostrar que el polinomio interpolante es único, consideremos dos polinomios $P_n(x)$ y $Q_n(x)$ que ambos interpola los mismos puntos:

$$P_n(x_i) = Q_n(x_i) = y_i, \quad i = 0, 1, \dots, n.$$

se define la función:

$$R(x) = P_n(x) - Q_n(x).$$

Entonces $R(x)$ es también un polinomio de grado a lo sumo n , y satisface:

$$R(x_i) = P_n(x_i) - Q_n(x_i) = 0, \quad \text{para todos } i = 0, 1, \dots, n.$$

Por lo tanto, $R(x)$ tiene $n + 1$ raíces distintas $x_0, x_1, \dots, x_n$. Sin embargo, un polinomio de grado n no puede tener más de n raíces distintas, a menos que todos sus coeficientes sean cero. Así,

$$R(x) \equiv 0 \implies P_n(x) = Q_n(x),$$

lo que demuestra que el polinomio interpolante es **único**.

Nota 4. ■ Para $n+1$ puntos distintos, siempre existe un polinomio de grado n que pasa exactamente por ellos.

- El cálculo directo mediante la matriz de Vandermonde es exacto teóricamente, pero numéricamente inestable para grandes n debido a la mala condición de la matriz.
- Por esta razón, en la práctica se prefieren formas más estables del polinomio, como las fórmulas de **Lagrange** o **Newton**.

Ejemplo 1. Supóngase que se tienen los puntos:

$$(1, 2), \quad (2, 3), \quad (4, 7).$$

El polinomio de grado 2 tendrá la forma:

$$P_2(x) = a_0 + a_1x + a_2x^2.$$

Sustituyendo los puntos:

$$\begin{cases} a_0 + a_1(1) + a_2(1)^2 = 2 \\ a_0 + a_1(2) + a_2(2)^2 = 3 \\ a_0 + a_1(4) + a_2(4)^2 = 7 \end{cases}$$

Al resolver el sistema, se obtiene:

$$a_0 = 1, \quad a_1 = 0.5, \quad a_2 = 0.25.$$

Por tanto,

$$P_2(x) = 1 + 0.5x + 0.25x^2.$$

Verificando:

$$P_2(1) = 2, \quad P_2(2) = 3, \quad P_2(4) = 7,$$

lo que confirma que el polinomio interpola correctamente los puntos.

Ejemplo 2. Interpolación los puntos:

$$(0, 1), \quad (1, 0), \quad (2, -1), \quad (3, 2).$$

Buscamos un polinomio cúbico:

$$P_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3.$$

Sustituyendo:

$$\begin{cases} a_0 = 1, \\ a_0 + a_1 + a_2 + a_3 = 0, \\ a_0 + 2a_1 + 4a_2 + 8a_3 = -1, \\ a_0 + 3a_1 + 9a_2 + 27a_3 = 2. \end{cases}$$

De la primera ecuación, $a_0 = 1$. Sustituyendo en las demás:

$$\begin{cases} a_1 + a_2 + a_3 = -1, \\ 2a_1 + 4a_2 + 8a_3 = -2, \\ 3a_1 + 9a_2 + 27a_3 = 1. \end{cases}$$

Resolviendo el sistema:

$$a_1 = -\frac{5}{2}, \quad a_2 = \frac{9}{4}, \quad a_3 = -\frac{3}{4}.$$

El polinomio interpolante es:

$$P_3(x) = 1 - \frac{5}{2}x + \frac{9}{4}x^2 - \frac{3}{4}x^3.$$

Verificando:

$$P_3(0) = 1, \quad P_3(1) = 0, \quad P_3(2) = -1, \quad P_3(3) = 2.$$

Interpretación: este polinomio de grado tres conecta los cuatro puntos de manera exacta. Si se grafican los datos y la curva, se observa un comportamiento ondulado típico de polinomios de orden alto, mostrando cómo la interpolación exacta puede oscilar entre los nodos.

Nota 5. ■ Para $n+1$ puntos distintos, existe un único polinomio de grado n que interpola los datos.

- El cálculo directo de los coeficientes mediante la matriz de Vandermonde es teóricamente exacto, pero numéricamente inestable para valores grandes de n .
- Por eficiencia y estabilidad se prefieren métodos como los de **Lagrange** y **Newton**, que se estudiarán a continuación.

1.3. Interpolación de Lagrange

Dado un conjunto de $n+1$ puntos con abscisas distintas

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n), \quad y_i = f(x_i),$$

el polinomio interpolante de Lagrange $P_n(x)$ se escribe como

$$P_n(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

Cada base L_i satisface $L_i(x_k) = \delta_{ik}$, por lo que $P_n(x_i) = y_i$ para todo i .

Si f es $(n+1)$ veces derivable en un intervalo que contiene a los nodos x_i y al punto x , entonces

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x), \quad \text{donde } \omega_{n+1}(x) = \prod_{i=0}^n (x - x_i),$$

para algún ξ en el intervalo convexo de $\{x_0, \dots, x_n, x\}$.

Nota 6. ■ Construir $P_n(x)$ y evaluarlo directamente con la fórmula de Lagrange cuesta $O(n^2)$ por punto de evaluación.

- Para muchos nodos o evaluaciones repetidas, se prefiere la forma baricéntrica por estabilidad y costo $O(n)$ por evaluación (se comenta al final).
- Con nodos equiespaciados y n grande puede aparecer el efecto Runge. Una mitigación es usar nodos de Chebyshev o splines.

Ejemplo 3. Interpolar $(0, 0)$, $(1, 1)$, $(2, 4)$ (muestran $f(x) = x^2$). Se obtiene

$$\begin{aligned} L_0(x) &= \frac{(x-1)(x-2)}{(0-1)(0-2)} = \frac{(x-1)(x-2)}{2}, \\ L_1(x) &= \frac{(x-0)(x-2)}{(1-0)(1-2)} = -(x)(x-2), \\ L_2(x) &= \frac{(x-0)(x-1)}{(2-0)(2-1)} = \frac{x(x-1)}{2}. \end{aligned}$$

Entonces

$$P_2(x) = 0 \cdot L_0(x) + 1 \cdot L_1(x) + 4 \cdot L_2(x) = x^2,$$

que recupera exactamente la función cuadrática.

1.4. Interpolación de Newton–Diferencias Finitas

El método de Newton construye el polinomio interpolante de manera incremental, usando el concepto de **diferencias divididas**. Esto permite agregar nuevos puntos sin recalcular todo el polinomio.

Sea un conjunto de $n + 1$ puntos distintos:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n).$$

El polinomio de Newton tiene la forma:

$$\begin{aligned} P_n(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots \\ &+ f[x_0, x_1, \dots, x_n] \prod_{j=0}^{n-1} (x - x_j), \end{aligned}$$

donde los coeficientes $f[x_i, \dots, x_j]$ son las *diferencias divididas*.

- De primer orden:

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}.$$

- De segundo orden:

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}.$$

- En general:

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Estas diferencias se organizan en una tabla triangular.

Ejemplo 4. *Interpolar los datos:*

$$(1, 2), (2, 3), (4, 7).$$

Paso 1: *construir la tabla de diferencias divididas:*

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$
1	2	$\frac{3-2}{2-1} = 1$	$\frac{(7-3)/(4-2)-1}{4-1} = \frac{1-1}{3} = 0$
2	3	$\frac{7-3}{4-2} = 2$	
4	7		

Paso 2: *polinomio de Newton:*

$$P_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1),$$

sustituyendo:

$$P_2(x) = 2 + 1(x - 1) + 0(x - 1)(x - 2) = 2 + (x - 1) = x + 1.$$

En este caso el término cuadrático se anuló (el polinomio resultante es lineal porque los puntos están casi alineados).

Ejemplo 5. Sea:

$$(0, 1), (1, 0), (2, -1), (3, 2).$$

Tabla de diferencias divididas:

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
0	1	$\frac{0-1}{1-0} = -1$	$\frac{(-1)-(-1)}{2-0} = 0$	$\frac{(1.5)-0}{3-0} = 0.5$
1	0	$\frac{-1-0}{2-1} = -1$	$\frac{2-(-1)}{3-1} = 1.5$	
2	-1	$\frac{2-(-1)}{3-2} = 3$		
3	2			

Polinomio:

$$\begin{aligned}
 P_3(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\
 &+ f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2) \\
 &= 1 - 1(x - 0) + 0(x)(x - 1) + 0.5(x)(x - 1)(x - 2).
 \end{aligned}$$

Simplificando:

$$P_3(x) = 1 - x + 0.5x(x - 1)(x - 2) = 1 - x + 0.5x^3 - 1.5x^2 + x = 1 - 1.5x^2 + 0.5x^3.$$

Por tanto:

$$P_3(x) = 1 - \frac{3}{2}x^2 + \frac{1}{2}x^3.$$

Verificando:

$$P_3(0) = 1, \quad P_3(1) = 0, \quad P_3(2) = -1, \quad P_3(3) = 2.$$

Nota 7. ■ Las diferencias divididas se pueden calcular una sola vez y reutilizar para múltiples evaluaciones.

- Si se añade un nuevo punto (x_{n+1}, y_{n+1}) , basta con agregar una nueva columna a la tabla sin recomputar todo.
- El polinomio de Newton es más estable que el de Lagrange para grandes n y se adapta mejor a la incorporación incremental de datos.

Ejemplo 6. Dados los puntos

$$(1, 2), (2, 3), (4, 7),$$

construyamos la tabla de diferencias divididas y el polinomio de Newton.

Paso 1: Primera columna (valores de la función).

$$f[x_0] = 2 \text{ (en } x_0 = 1), \quad f[x_1] = 3 \text{ (en } x_1 = 2), \quad f[x_2] = 7 \text{ (en } x_2 = 4).$$

Paso 2: Diferencias divididas de primer orden.

$$\begin{aligned}
 f[x_0, x_1] &= \frac{f[x_1] - f[x_0]}{x_1 - x_0} = \frac{3 - 2}{2 - 1} = 1, \\
 f[x_1, x_2] &= \frac{f[x_2] - f[x_1]}{x_2 - x_1} = \frac{7 - 3}{4 - 2} = 2.
 \end{aligned}$$

Paso 3: Diferencia dividida de segundo orden.

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{2 - 1}{4 - 1} = \frac{1}{3}.$$

Atencion: En el ejemplo del texto principal se obtuvo cero por un redondeo al mostrar pasos; aquí dejamos el valor exacto $\frac{1}{3}$. Para verificar coherencia, construyamos el polinomio y validemos en los nodos.

Paso 4: Polinomio de Newton.

$$\begin{aligned} P_2(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ &= 2 + 1(x - 1) + \frac{1}{3}(x - 1)(x - 2). \end{aligned}$$

Paso 5: Verificacion en los nodos.

$$\begin{aligned} P_2(1) &= 2 + 1(0) + \frac{1}{3}(0)(-1) = 2, \\ P_2(2) &= 2 + 1(1) + \frac{1}{3}(1)(0) = 3, \\ P_2(4) &= 2 + 1(3) + \frac{1}{3}(3)(2) = 2 + 3 + 2 = 7. \end{aligned}$$

Paso 6: Evaluacion en un punto intermedio (por ejemplo, $x = 3$).

$$P_2(3) = 2 + 1(2) + \frac{1}{3}(2)(1) = 2 + 2 + \frac{2}{3} = \frac{14}{3}.$$

Tabla triangular resumida.

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$
1	2	1	$\frac{1}{3}$
2	3	2	
4	7		

Ejemplo 7. *Dados los puntos*

$$(0, 1), \quad (1, 0), \quad (2, -1), \quad (3, 2),$$

construyamos la tabla y el polinomio $P_3(x)$.

Paso 1: Primera columna.

$$f[x_0] = 1, \quad f[x_1] = 0, \quad f[x_2] = -1, \quad f[x_3] = 2.$$

Paso 2: Primer orden.

$$\begin{aligned} f[x_0, x_1] &= \frac{0 - 1}{1 - 0} = -1, \\ f[x_1, x_2] &= \frac{-1 - 0}{2 - 1} = -1, \\ f[x_2, x_3] &= \frac{2 - (-1)}{3 - 2} = 3. \end{aligned}$$

Paso 3: Segundo orden.

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{(-1) - (-1)}{2 - 0} = 0,$$

$$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1} = \frac{3 - (-1)}{3 - 1} = \frac{4}{2} = 2.$$

Paso 4: Tercer orden.

$$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0} = \frac{2 - 0}{3 - 0} = \frac{2}{3}.$$

Paso 5: Polinomio de Newton.

$$\begin{aligned} P_3(x) &= f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ &\quad + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2) \\ &= 1 + (-1)(x - 0) + 0 \cdot (x)(x - 1) + \frac{2}{3}(x)(x - 1)(x - 2). \end{aligned}$$

Paso 6: Verificación en los nodos.

$$\begin{aligned} P_3(0) &= 1 - 0 + 0 + 0 = 1, \\ P_3(1) &= 1 - 1 + 0 + 0 = 0, \\ P_3(2) &= 1 - 2 + 0 + 0 = -1, \\ P_3(3) &= 1 - 3 + 0 + \frac{2}{3}(3)(2)(1) = -2 + 4 = 2. \end{aligned}$$

Paso 7: Forma expandida (opcional para comparación).

$$\begin{aligned} P_3(x) &= 1 - x + \frac{2}{3}(x^3 - 3x^2 + 2x) \\ &= 1 - x + \frac{2}{3}x^3 - 2x^2 + \frac{4}{3}x \\ &= 1 - 2x^2 + \frac{2}{3}x^3 + \frac{1}{3}x. \end{aligned}$$

(La forma de Newton es preferible para evaluación incremental y estabilidad estructural.)

Tabla triangular completa.

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$f[x_i, x_{i+1}, x_{i+2}, x_{i+3}]$
0	1	-1	0	$\frac{2}{3}$
1	0	-1	2	
2	-1	3		
3	2			

Paso 8: Evaluación en un punto interno (por ejemplo, $x = 1.5$).

$$\begin{aligned} P_3(1.5) &= 1 + (-1)(1.5) + 0 \cdot (1.5)(0.5) + \frac{2}{3}(1.5)(0.5)(-0.5) \\ &= 1 - 1.5 + \frac{2}{3} \cdot (1.5 \cdot 0.5 \cdot -0.5) \\ &= -0.5 + \frac{2}{3} \cdot (-0.375) \\ &= -0.5 - 0.25 = -0.75. \end{aligned}$$

Ejemplo 8. $f(x) = x^2$ en $x_0 = 0$, $h = 1$, estimar $f(0.5)$ Datos en $x = 0, 1, 2, 3$:

x	0	1	2	3
$y = f(x) = x^2$	0	1	4	9

Tabla de diferencias progresivas (en la primera fila):

$$\Delta y_0 = 1 - 0 = 1 \quad \Delta^2 y_0 = 3 - 1 = 2 \quad \Delta^3 y_0 = 2 - 2 = 0$$

Aquí $\Delta y_1 = 4 - 1 = 3$, $\Delta^2 y_1 = 5 - 3 = 2$ (mostradas sólo para cálculo de $\Delta^2 y_0$).

$$\text{Parámetro } p = \frac{x - x_0}{h} = \frac{0.5 - 0}{1} = 0.5.$$

$$\begin{aligned} P(0.5) &= y_0 + p \Delta y_0 + \frac{p(p-1)}{2} \Delta^2 y_0 \\ &= 0 + (0.5)(1) + \frac{0.5(-0.5)}{2} (2) \\ &= 0.5 + \left(-\frac{0.25}{2}\right) (2) = 0.5 - 0.25 = 0.25. \end{aligned}$$

Valor exacto: $f(0.5) = 0.25$. Coincide (el término de tercer orden es nulo).

Ejemplo 9. $f(x) = x^3$ en $x_0 = 1$, $h = 1$, estimar $f(1.5)$ Datos en $x = 1, 2, 3, 4$:

x	1	2	3	4
$y = f(x) = x^3$	1	8	27	64

Diferencias progresivas (primera fila en $x_0 = 1$):

$$\Delta y_0 = 8 - 1 = 7, \quad \Delta y_1 = 27 - 8 = 19, \quad \Delta y_2 = 64 - 27 = 37.$$

$$\Delta^2 y_0 = 19 - 7 = 12, \quad \Delta^2 y_1 = 37 - 19 = 18, \quad \Delta^3 y_0 = 18 - 12 = 6.$$

$$\text{Parámetro } p = \frac{x - x_0}{h} = \frac{1.5 - 1}{1} = 0.5.$$

$$\begin{aligned} P(1.5) &= y_0 + p \Delta y_0 + \frac{p(p-1)}{2} \Delta^2 y_0 + \frac{p(p-1)(p-2)}{6} \Delta^3 y_0 \\ &= 1 + (0.5)(7) + \frac{0.5(-0.5)}{2} (12) + \frac{0.5(-0.5)(-1.5)}{6} (6). \end{aligned}$$

Cálculo término a término:

$$1 + 3.5 + \left(\frac{-0.25}{2}\right) 12 + \left(\frac{0.375}{6}\right) 6 = 1 + 3.5 - 1.5 + 0.375 = 3.375.$$

Exacto: $f(1.5) = (1.5)^3 = 3.375$.

Ejemplo 10. $f(x) = x^2$ en $x_n = 3$, $h = 1$, estimar $f(2.6)$ Datos en $x = 0, 1, 2, 3$ (como en el Ejemplo 1). Diferencias regresivas en el extremo $x_3 = 3$:

$$\nabla y_3 = y_3 - y_2 = 9 - 4 = 5, \quad \nabla^2 y_3 = \nabla y_3 - \nabla y_2 = 5 - 3 = 2, \quad \nabla^3 y_3 = 2 - 2 = 0.$$

Aquí $\nabla y_2 = y_2 - y_1 = 3$, $\nabla^2 y_2 = \nabla y_2 - \nabla y_1 = 3 - 2 = 1$ (solo de apoyo).

$$\text{Parámetro } q = \frac{x - x_n}{h} = \frac{2.6 - 3}{1} = -0.4.$$

$$\begin{aligned} P(2.6) &= y_3 + q \nabla y_3 + \frac{q(q+1)}{2} \nabla^2 y_3 \\ &= 9 + (-0.4)(5) + \frac{(-0.4)(0.6)}{2}(2) \\ &= 9 - 2 - 0.24 = 6.76. \end{aligned}$$

$$\text{Exacto: } f(2.6) = (2.6)^2 = 6.76.$$

Ejemplo 11. $f(x) = x^3$ en $x_n = 4$, $h = 1$, estimar $f(3.2)$ Datos en $x = 1, 2, 3, 4$. Diferencias regresivas en el extremo $x_4 = 4$:

$$\begin{aligned} \nabla y_4 &= y_4 - y_3 = 64 - 27 = 37, \\ \nabla^2 y_4 &= \nabla y_4 - \nabla y_3 = 37 - 19 = 18, \\ \nabla^3 y_4 &= \nabla^2 y_4 - \nabla^2 y_3 = 18 - 12 = 6. \end{aligned}$$

$$\text{Parámetro } q = \frac{x - x_n}{h} = \frac{3.2 - 4}{1} = -0.8.$$

$$\begin{aligned} P(3.2) &= y_4 + q \nabla y_4 + \frac{q(q+1)}{2} \nabla^2 y_4 + \frac{q(q+1)(q+2)}{6} \nabla^3 y_4 \\ &= 64 + (-0.8)(37) + \frac{(-0.8)(0.2)}{2}(18) + \frac{(-0.8)(0.2)(1.2)}{6}(6). \end{aligned}$$

Cálculo:

$$64 - 29.6 + (-0.08)(18) + \left(\frac{-0.192}{6}\right)6 = 64 - 29.6 - 1.44 - 0.192 = 32.768.$$

$$\text{Exacto: } f(3.2) = (3.2)^3 = 32.768.$$

Nota 8. ■ Use Newton hacia adelante si x está próximo a x_0 ; use Newton hacia atrás si x está próximo a x_n .

- Para funciones polinómicas de grado m , las diferencias de orden $> m$ son cero y la fórmula se vuelve exacta con $m + 1$ nodos.
- En datos reales con ruido, truncar la serie a pocas diferencias suele mejorar estabilidad.

Nota 9. Cuando los nodos x_i están igualmente espaciados, es posible expresar el polinomio interpolante usando **diferencias finitas**. Estas fórmulas son variantes del método de Newton y se dividen en dos casos:

- **Newton hacia adelante:** se usa cuando el punto donde se desea interpolar está cerca del primer nodo x_0 .
- **Newton hacia atrás:** se usa cuando el punto está cerca del último nodo x_n .

Sea el conjunto de nodos equiespaciados:

$$x_i = x_0 + i h, \quad i = 0, 1, \dots, n, \quad h > 0.$$

Las diferencias finitas se definen como:

$$\begin{aligned}\Delta y_i &= y_{i+1} - y_i, \\ \Delta^2 y_i &= \Delta(\Delta y_i) = \Delta y_{i+1} - \Delta y_i, \\ \Delta^3 y_i &= \Delta(\Delta^2 y_i) = \Delta^2 y_{i+1} - \Delta^2 y_i, \quad \text{etc.}\end{aligned}$$

Y de forma análoga, las diferencias regresivas:

$$\begin{aligned}\nabla y_i &= y_i - y_{i-1}, \\ \nabla^2 y_i &= \nabla y_i - \nabla y_{i-1}, \quad \text{etc.}\end{aligned}$$

Newton hacia adelante:

$$P(x) = y_0 + p \Delta y_0 + \frac{p(p-1)}{2!} \Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!} \Delta^3 y_0 + \dots,$$

$$\text{donde } p = \frac{x - x_0}{h}.$$

Newton hacia atrás:

$$P(x) = y_n + q \nabla y_n + \frac{q(q+1)}{2!} \nabla^2 y_n + \frac{q(q+1)(q+2)}{3!} \nabla^3 y_n + \dots,$$

$$\text{donde } q = \frac{x - x_n}{h}.$$

Ejercicio 1.

x	0	1	2	3
$f(x)$	0	1	4	9

Cálculo de diferencias progresivas:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
0	0	1	2	0
1	1	3	2	
2	4	5		
3	9			

Queremos estimar: $f(0.5)$, con $x_0 = 0$, $h = 1$ y $p = 0.5$.

Sustitución:

$$\begin{aligned}P(0.5) &= y_0 + p \Delta y_0 + \frac{p(p-1)}{2!} \Delta^2 y_0 \\ &= 0 + (0.5)(1) + \frac{(0.5)(-0.5)}{2}(2) \\ &= 0.5 - 0.25 = 0.25.\end{aligned}$$

Verificación: $f(0.5) = (0.5)^2 = 0.25. \Rightarrow$ Coincide exactamente.

Ejercicio 2.

x	1	2	3	4
$f(x)$	1	8	27	64

Tabla de diferencias:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$
1	1	7	12	6
2	8	19	18	
3	27	37		
4	64			

Queremos estimar: $f(1.5)$. $x_0 = 1$, $h = 1$, $p = 0.5$.

$$\begin{aligned}
 P(1.5) &= y_0 + p \Delta y_0 + \frac{p(p-1)}{2!} \Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!} \Delta^3 y_0 \\
 &= 1 + (0.5)(7) + \frac{(0.5)(-0.5)}{2} (12) + \frac{(0.5)(-0.5)(-1.5)}{6} (6) \\
 &= 1 + 3.5 - 1.5 + 0.375 = 3.375.
 \end{aligned}$$

Verificación: $f(1.5) = (1.5)^3 = 3.375$.

Conclusión: el método reproduce perfectamente el valor porque f es un polinomio de grado 3 y se usaron 4 nodos.

Ejercicio 3.

x	0	1	2	3
$f(x)$	0	1	4	9

Diferencias regresivas (desde el final):

x	y	∇y	$\nabla^2 y$	$\nabla^3 y$
0	0			
1	1	1		
2	4	3	2	
3	9	5	2	0

Queremos estimar: $f(2.6)$. $x_n = 3$, $h = 1$, $q = \frac{2.6 - 3}{1} = -0.4$.

$$\begin{aligned}
 P(2.6) &= y_3 + q \nabla y_3 + \frac{q(q+1)}{2!} \nabla^2 y_3 \\
 &= 9 + (-0.4)(5) + \frac{(-0.4)(0.6)}{2} (2) \\
 &= 9 - 2 - 0.24 = 6.76.
 \end{aligned}$$

Verificación: $f(2.6) = (2.6)^2 = 6.76$.

Ejercicio 4.

x	1	2	3	4
$f(x)$	1	8	27	64

Diferencias regresivas:

x	y	∇y	$\nabla^2 y$	$\nabla^3 y$
1	1			
2	8	7		
3	27	19	12	
4	64	37	18	6

Queremos estimar: $f(3.2)$. $x_n = 4$, $h = 1$, $q = \frac{3.2 - 4}{1} = -0.8$.

$$\begin{aligned} P(3.2) &= y_4 + q \nabla y_4 + \frac{q(q+1)}{2!} \nabla^2 y_4 + \frac{q(q+1)(q+2)}{3!} \nabla^3 y_4 \\ &= 64 + (-0.8)(37) + \frac{(-0.8)(0.2)}{2} (18) + \frac{(-0.8)(0.2)(1.2)}{6} (6). \end{aligned}$$

Cálculo numérico:

$$64 - 29.6 - 1.44 - 0.192 = 32.768.$$

Verificación: $f(3.2) = (3.2)^3 = 32.768$.

Conclusión: el polinomio reproduce exactamente la función cúbica con 4 nodos.

Nota 10. ■ Las fórmulas hacia adelante y hacia atrás son exactas para funciones polinómicas de grado $\leq n$ si se utilizan $n + 1$ nodos.

- Si x está cerca del inicio de la tabla, use Newton hacia adelante; si está cerca del final, use Newton hacia atrás.
- Las diferencias de orden superior tienden a cero si f es suave; truncar la serie cuando $\Delta^k y_i$ o $\nabla^k y_i$ son pequeños mejora estabilidad.

Ejemplo 12. Implementación numérica:

```
is_equispaced <- function(x, tol = 1e-9) {
  if (length(x) < 2) return(TRUE)
  h <- diff(x)
  max(abs(h - h[1])) < tol
}

falling_prod <- function(p, k) {
  if (k == 0) return(1)
  prod(p - 0:(k-1))
}

rising_prod <- function(q, k) {
  if (k == 0) return(1)
  prod(q + 0:(k-1))
}

# =====
```

```

# Tablas de diferencias
# =====
forward_diff_table <- function(x, y, check_equispaced = TRUE) {
  n <- length(x)
  if (length(y) != n) stop("x e y deben tener la misma longitud.")
  if (check_equispaced && !is_equispaced(x)) stop("Los nodos no son equiespaciados.")
  T <- matrix(0, nrow = n, ncol = n)
  T[,1] <- y
  if (n >= 2) {
    for (j in 2:n) {
      for (i in 1:(n - j + 1)) {
        T[i,j] <- T[i+1,j-1] - T[i,j-1]    # Delta^j y_i
      }
    }
  }
  T
}

backward_diff_table <- function(x, y, check_equispaced = TRUE) {
  n <- length(x)
  if (length(y) != n) stop("x e y deben tener la misma longitud.")
  if (check_equispaced && !is_equispaced(x)) stop("Los nodos no son equiespaciados.")
  T <- matrix(0, nrow = n, ncol = n)
  T[,1] <- y
  if (n >= 2) {
    for (j in 2:n) {
      for (i in n:j) {
        T[i,j] <- T[i,j-1] - T[i-1,j-1]    # Nabla^j y_i
      }
    }
  }
  T
}

# =====
# Evaluadores Newton
# - max_order: orden maximo a usar (por defecto, n-1). Truncar ayuda con ruido.
# =====
newton_forward_eval <- function(x, T, xq, max_order = NULL) {
  n <- length(x)
  h <- x[2] - x[1]
  p <- (xq - x[1]) / h
  if (is.null(max_order)) max_order <- n - 1
  max_order <- max(0, min(max_order, n - 1))
  P <- T[1,1]
  for (k in 1:max_order) {
    coef <- falling_prod(p, k) / factorial(k)
    P <- P + coef * T[1, k + 1]
  }
}

```

```

}
# Estimacion de error de truncamiento (termino siguiente) si existe:
err <- NA_real_
next_k <- max_order + 1
if (next_k <= n - 1) {
  coef_next <- falling_prod(p, next_k) / factorial(next_k)
  err <- abs(coef_next * T[1, next_k + 1])
}
list(value = P, est_trunc_error = err, p = p)
}

newton_backward_eval <- function(x, T, xq, max_order = NULL) {
  n <- length(x)
  h <- x[2] - x[1]
  q <- (xq - x[n]) / h
  if (is.null(max_order)) max_order <- n - 1
  max_order <- max(0, min(max_order, n - 1))
  P <- T[n,1]
  for (k in 1:max_order) {
    coef <- rising_prod(q, k) / factorial(k)
    P <- P + coef * T[n, k + 1]
  }
  # Estimacion de error (termino siguiente) si existe:
  err <- NA_real_
  next_k <- max_order + 1
  if (next_k <= n - 1) {
    coef_next <- rising_prod(q, next_k) / factorial(next_k)
    err <- abs(coef_next * T[n, next_k + 1])
  }
  list(value = P, est_trunc_error = err, q = q)
}

# =====
# Seleccion automatica de metodo y evaluacion vectorizada
# - method = "auto" | "forward" | "backward"
# - max_order: truncamiento del orden
# Devuelve data.frame con resultados por cada xq
# =====
newton_interp_equispaced <- function(x, y, xq, method = "auto", max_order = NULL) {
  if (!is_equispaced(x)) stop("Los nodos x deben ser equiespaciados.")
  n <- length(x)
  if (length(y) != n) stop("x e y deben tener la misma longitud.")
  # Precomputo de tablas
  Tforw <- forward_diff_table(x, y, check_equispaced = FALSE)
  Tback <- backward_diff_table(x, y, check_equispaced = FALSE)
  # Funcion auxiliar para elegir metodo segun cercania
  choose_method <- function(xq_single) {
    if (method == "forward") return("forward")

```

```

    if (method == "backward") return("backward")
    # auto: decide por cercania
    d0 <- abs(xq_single - x[1])
    dn <- abs(xq_single - x[n])
    if (d0 <= dn) "forward" else "backward"
  }
# Evaluacion (vectorizada)
vals <- numeric(length(xq))
errs <- rep(NA_real_, length(xq))
meth <- character(length(xq))
par1 <- numeric(length(xq)) # p o q usado
for (i in seq_along(xq)) {
  m <- choose_method(xq[i])
  meth[i] <- m
  if (m == "forward") {
    res <- newton_forward_eval(x, Tforw, xq[i], max_order = max_order)
    vals[i] <- res$value
    errs[i] <- res$est_trunc_error
    par1[i] <- res$p
  } else {
    res <- newton_backward_eval(x, Tback, xq[i], max_order = max_order)
    vals[i] <- res$value
    errs[i] <- res$est_trunc_error
    par1[i] <- res$q
  }
}
}
data.frame(
  xq = xq,
  estimate = vals,
  est_trunc_error = errs,
  method_used = meth,
  p_or_q = par1
)
}

# =====
# Ejemplos rapidos
# =====
# Ejemplo 1 (adelante, auto): f(x)=x^2 en x=0:3, evaluar en xq=seq(0,2,by=0.5)
# x <- 0:3; y <- x^2
# newton_interp_equispaced(x, y, seq(0, 2, by = 0.5))

# Ejemplo 2 (atras, auto): f(x)=x^3 en x=1:4, evaluar en xq=c(3.2, 3.6)
# x <- 1:4; y <- x^3
# newton_interp_equispaced(x, y, c(3.2, 3.6))

```

1.5. Splines

Los **splines** son funciones polinómicas por tramos utilizadas para interpolar un conjunto de puntos de forma suave y estable. A diferencia de un único polinomio de grado alto (que puede oscilar violentamente entre nodos, efecto de Runge), los splines utilizan polinomios de bajo grado en cada intervalo, garantizando continuidad en los nodos y sus derivadas. Le idea es que en cada subintervalo $[x_i, x_{i+1}]$ se construye un polinomio cúbico $S_i(x)$, y se imponen condiciones para que toda la función $S(x)$ sea continua, con derivadas suaves.

Definición 2. Sea un conjunto de nodos: $x_0 < x_1 < \dots < x_n$, y valores conocidos: $y_i = f(x_i)$, $i = 0, 1, \dots, n$.

El **spline cúbico** $S(x)$ se define por tramos:

$$S(x) = \begin{cases} S_0(x), & x_0 \leq x < x_1, \\ S_1(x), & x_1 \leq x < x_2, \\ \vdots \\ S_{n-1}(x), & x_{n-1} \leq x \leq x_n, \end{cases} \quad (4)$$

donde cada tramo es un polinomio cúbico:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3. \quad (5)$$

El spline cúbico debe cumplir:

1. Interpolación: $S_i(x_i) = y_i$, $S_i(x_{i+1}) = y_{i+1}$.
2. Continuidad de la primera derivada: $S'_i(x_{i+1}) = S'_{i+1}(x_{i+1})$.
3. Continuidad de la segunda derivada: $S''_i(x_{i+1}) = S''_{i+1}(x_{i+1})$.

Estas condiciones producen $4(n - 1)$ ecuaciones, que junto con dos condiciones adicionales en los extremos definen completamente los $4n$ coeficientes.

- **Spline natural:** Se impone $S''(x_0) = S''(x_n) = 0$. Esto genera una forma “suave” en los extremos.
- **Spline completo (o clamped):** Se imponen las derivadas primeras conocidas: $S'(x_0) = f'(x_0)$, $S'(x_n) = f'(x_n)$.
- **Spline “not-a-knot”:** Obliga continuidad de la tercera derivada en los nodos interiores x_1 y x_{n-1} .

Sistema tridiagonal de las segundas derivadas

Definimos:

$$h_i = x_{i+1} - x_i, \quad m_i = \frac{y_{i+1} - y_i}{h_i}, \quad i = 0, \dots, n - 1. \quad (6)$$

El sistema para las segundas derivadas $M_i = S''(x_i)$ es:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6(m_i - m_{i-1}), \quad i = 1, \dots, n - 1, \quad (7)$$

con las condiciones de frontera:

$$\text{Spline natural: } M_0 = M_n = 0. \quad (8)$$

Una vez calculadas las M_i , cada tramo $S_i(x)$ se obtiene como:

$$S_i(x) = \frac{M_{i+1}(x - x_i)^3}{6h_i} - \frac{M_i(x_{i+1} - x)^3}{6h_i} + \left(\frac{y_{i+1}}{h_i} - \frac{M_{i+1}h_i}{6} \right) (x - x_i) + \left(\frac{y_i}{h_i} - \frac{M_ih_i}{6} \right) (x_{i+1} - x). \quad (9)$$

Algoritmo 1. El sistema tridiagonal se resuelve eficientemente con el **método de Thomas** (algoritmo especializado para matrices tridiagonales). El proceso computacional sigue estos pasos:

1. Calcular los h_i y m_i .
2. Formar el sistema tridiagonal en términos de M_i .
3. Resolver para obtener $M_1, \dots, M_{n-1}$.
4. Construir los polinomios $S_i(x)$ en cada subintervalo.

Ejemplo 13. Interpolar los puntos:

$$(0, 0), \quad (1, 1), \quad (2, 0).$$

Se tiene $n = 2$, $h_0 = h_1 = 1$, $m_0 = 1$, $m_1 = -1$.

Ecuación central:

$$h_0M_0 + 2(h_0 + h_1)M_1 + h_1M_2 = 6(m_1 - m_0),$$

$$1(0) + 4M_1 + 1(0) = 6(-1 - 1) = -12 \quad \Rightarrow \quad M_1 = -3.$$

$$Y \quad M_0 = M_2 = 0.$$

Sustituyendo en la fórmula del tramo $[0, 1]$:

$$S_0(x) = -\frac{M_1}{6}(x - 1)^3 + x.$$

En $[1, 2]$:

$$S_1(x) = -\frac{M_1}{6}(2 - x)^3 + (2 - x).$$

Graficar ambos tramos muestra una curva suave en forma de “loma”.

Ejemplo 14. Puntos:

$$(0, 0), \quad (1, 1), \quad (2, 0), \quad (3, 1).$$

Datos: $h_i = 1$, $m_0 = 1$, $m_1 = -1$, $m_2 = 1$.

Sistema:

$$\begin{cases} 4M_1 + M_2 = 6(-1 - 1) = -12, \\ M_1 + 4M_2 = 6(1 - (-1)) = 12. \end{cases}$$

Resolviendo:

$$M_1 = -4, \quad M_2 = 4, \quad M_0 = M_3 = 0.$$

Cada tramo se obtiene sustituyendo los valores de M_i en la fórmula general.

Ejemplo 15. *Puntos:*

$$(0, 1), \quad (1, 2), \quad (2, 0),$$

con derivadas conocidas $S'(0) = 0$ y $S'(2) = -1$.

Se aplican las condiciones en los extremos:

$$2h_0M_0 + h_0M_1 = 6 \left(\frac{y_1 - y_0}{h_0} - S'(0) \right),$$

$$h_0M_0 + 2h_0M_1 = 6 \left(S'(2) - \frac{y_2 - y_1}{h_0} \right).$$

Resolviendo se obtienen M_0 y M_1 , y luego los coeficientes a_i, b_i, c_i, d_i de cada tramo.

Ejemplo 16. *Mediciones de temperatura ($^{\circ}C$) a distintas horas:*

Hora (h)	0	3	6	9
Temperatura	15	18	14	10

Interpolan con un spline cúbico natural y estimar $T(4.5)$.

Algoritmo 2. 1. Calcular $h_i = 3$, $m_i = (y_{i+1} - y_i)/h_i$.

2. Formar el sistema para M_1, M_2 .

3. Resolverlo con el método de Thomas.

4. Evaluar $S_1(4.5)$ usando la fórmula de $S_i(x)$ en el intervalo $[3, 6]$.

Resultado esperado: $T(4.5) \approx 15.5^{\circ}C$. La curva obtenida describe de forma suave la variación de temperatura entre mediciones.

Nota 11. ■ Los splines cúbicos son continuos en S , S' , y S'' , lo que garantiza suavidad.

- Reducen el error sin introducir oscilaciones (efecto Runge).
- El error máximo en un spline cúbico natural es proporcional a $O(h^4)$.
- Son ampliamente usados en gráficos por computadora, CAD, animación y análisis de datos experimentales.

1.6. Error de Interpolación

La interpolación polinómica busca aproximar una función $f(x)$ mediante un polinomio $P_n(x)$ que coincide con $f(x)$ en $n+1$ puntos conocidos. Sin embargo, entre los nodos de interpolación puede existir una diferencia o **error de interpolación**, denotado por $R_n(x)$.

Fórmula general del error

Sea $f \in C^{(n+1)}[a, b]$ y $P_n(x)$ el polinomio interpolante de grado n para los puntos $(x_0, y_0), \dots, (x_n, y_n)$ con $y_i = f(x_i)$. Entonces, para todo $x \in [a, b]$, existe un número $\xi \in (a, b)$ tal que:

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i).$$

Interpretación: El error depende de dos factores:

- El término $\frac{f^{(n+1)}(\xi)}{(n+1)!}$, relacionado con la suavidad de $f(x)$.
- El producto $\prod_{i=0}^n (x - x_i)$, que refleja la posición de los nodos de interpolación.

Cota del error

Si se conoce una cota $|f^{(n+1)}(x)| \leq M$ para $x \in [a, b]$, entonces:

$$|R_n(x)| \leq \frac{M}{(n+1)!} \max_{x \in [a, b]} \left| \prod_{i=0}^n (x - x_i) \right|.$$

Esta expresión indica que el error aumenta:

- Si $f^{(n+1)}(x)$ es grande (función muy curvada o con derivadas altas).
- Si los nodos x_i están mal distribuidos (especialmente equiespaciados para n grande).

Comportamiento del error y efecto de Runge

Cuando los nodos son equiespaciados, el término

$$\prod_{i=0}^n (x - x_i)$$

crece rápidamente en los extremos del intervalo, provocando oscilaciones en el polinomio interpolante — fenómeno conocido como **efecto de Runge**. Este efecto se agrava con n grande y funciones con alta curvatura, como $f(x) = \frac{1}{1+x^2}$.

Conclusión: aumentar el grado n no garantiza una mejor aproximación.

Nodos de Chebyshev (minimizan el error máximo)

Para reducir las oscilaciones, se eligen nodos no equiespaciados, conocidos como **nodos de Chebyshev**. En el intervalo $[-1, 1]$ se definen como:

$$x_i = \cos\left(\frac{2i+1}{2(n+1)}\pi\right), \quad i = 0, 1, \dots, n.$$

Estos nodos distribuyen más puntos cerca de los extremos del intervalo, donde el error tiende a crecer más.

Propiedad fundamental: Los nodos de Chebyshev minimizan la cantidad

$$\max_{x \in [-1, 1]} \left| \prod_{i=0}^n (x - x_i) \right|,$$

y, por tanto, minimizan el error máximo de interpolación en el sentido de la norma infinita.

Ejemplo 17. : *comparación cualitativa*

Sea $f(x) = \frac{1}{1+x^2}$ en $[-5, 5]$.

- Con $n = 10$ y nodos **equiespaciados**: el polinomio presenta grandes oscilaciones en los extremos (efecto de Runge).
- Con $n = 10$ y nodos **de Chebyshev**: la interpolación es estable y el error máximo es mucho menor.

Ejemplo 18. *Estimación del error* Sea $f(x) = e^x$ en $[0, 1]$, con nodos equiespaciados $x_0 = 0$, $x_1 = 0.5$, $x_2 = 1$. Entonces $f^{(3)}(x) = e^x \leq e$.

Usando la fórmula del error para $x = 0.25$:

$$|R_2(0.25)| \leq \frac{e}{3!} |(0.25 - 0)(0.25 - 0.5)(0.25 - 1)| = \frac{e}{6} (0.25 \cdot 0.25 \cdot 0.75) \approx 0.0085.$$

La cota muestra que el error es pequeño, y decrece proporcionalmente a h^{n+1} .

Nota 12. ■ La magnitud de $R_n(x)$ depende tanto de la función como de la elección de nodos.

- Nodos de Chebyshev distribuyen mejor el error, especialmente para grados altos.
- En la práctica, cuando los datos son experimentales o $f(x)$ es desconocida, se prefiere reducir n y usar **splines**, que mantienen continuidad y baja oscilación.

1.7. Resumen

La **interpolación** es una de las herramientas fundamentales del análisis numérico. Su objetivo es aproximar una función desconocida $f(x)$ a partir de un conjunto discreto de datos

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n),$$

buscando una función (generalmente un polinomio) que pase exactamente por esos puntos.

Existencia y unicidad. Para $n+1$ nodos distintos, existe un **único polinomio** $P_n(x)$ de grado menor o igual que n tal que $P_n(x_i) = y_i$. Su unicidad se demuestra mediante la no singularidad de la *matriz de Vandermonde*, garantizando que el sistema lineal asociado tenga una única solución.

Formas del polinomio interpolante. El mismo polinomio puede escribirse en distintas formas algebraicamente equivalentes:

- **Forma de Lagrange:** usa funciones base $L_i(x)$ independientes, útiles para casos con pocos nodos.
- **Forma de Newton:** emplea *diferencias divididas* y permite actualizar el polinomio fácilmente al añadir nuevos datos.
- **Formas de diferencias finitas:** (hacia adelante o hacia atrás) aplicables cuando los nodos son equiespaciados, optimizan la evaluación cerca de los extremos.
- **Interpolación por tramos (splines):** utiliza polinomios de bajo grado en cada intervalo, garantizando continuidad y suavidad, y evitando las oscilaciones de alto grado.

Error de interpolación. La diferencia entre la función real y el polinomio interpolante está dada por:

$$R_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

para algún $\xi \in (a, b)$. El error depende de la suavidad de $f(x)$ (a través de su derivada de orden $n+1$) y de la distribución de los nodos. Cuando los nodos son equiespaciados y n es grande, el polinomio puede oscilar fuertemente en los extremos, fenómeno conocido como **efecto de Runge**. Este problema se reduce empleando **nodos de Chebyshev**, que minimizan el error máximo, o mediante el uso de **splines cúbicos**, que suavizan la interpolación.

Recomendaciones prácticas.

- **Pocos nodos:** la forma de Lagrange es simple y directa.
- **Muchos nodos o evaluaciones repetidas:** la forma de Newton es más eficiente.
- **Datos con ruido o alta oscilación:** conviene reducir el grado del polinomio, usar nodos de Chebyshev o preferir splines.
- **Tablas equiespaciadas:** las fórmulas de diferencias finitas permiten interpolar y derivar valores de manera rápida.

La interpolación no sólo permite reconstruir funciones a partir de datos discretos, sino que también constituye la base de numerosas aplicaciones en ingeniería, estadística, modelado y simulación numérica. Los métodos de **Lagrange**, **Newton** y **splines** forman el núcleo de técnicas más avanzadas, como la aproximación por mínimos cuadrados, el ajuste de curvas suavizadas y las representaciones polinomiales adaptativas.

En síntesis: la interpolación combina teoría algebraica, estabilidad numérica y diseño de nodos, buscando un equilibrio entre precisión, simplicidad y robustez computacional.

1.8. Ejercicios

Esta sección presenta ejercicios orientados a consolidar la comprensión de los métodos de interpolación estudiados: Lagrange, Newton (diferencias divididas) y Newton hacia adelante/atrás (diferencias finitas).

Se recomienda resolver los primeros ejercicios de forma manual y posteriormente verificar los resultados con el código en R proporcionado.

Ejercicio 5. 1. Dado el conjunto de puntos

$$(0, 1), (1, 3), (2, 11),$$

- a) Determine el polinomio interpolante mediante el **método de Lagrange**.
- b) Verifique el resultado construyendo la **tabla de diferencias divididas** y aplicando el **método de Newton**.
- c) Evalúe el polinomio en $x = 1.5$ y compare con la función cuadrática ajustada visualmente a los puntos.

2. Con los puntos

$$(1, 2), (2, 5), (3, 10), (4, 17),$$

- a) Obtenga el polinomio interpolante de grado 3 usando el **método de Newton**.
- b) Muestre la tabla completa de diferencias divididas.
- c) Evalúe el polinomio en $x = 2.5$.
- d) Verifique el resultado calculando directamente $f(x) = x^2 + 1$ y compare.

3. Dados los puntos

$$(-1, 4), (0, 1), (2, 3),$$

- a) Construya el polinomio de Lagrange.
- b) Expanda y simplifique el resultado hasta la forma general $P_2(x) = a_0 + a_1x + a_2x^2$.
- c) Verifique los valores de $P_2(-1)$, $P_2(0)$ y $P_2(2)$.

Ejercicio 6. 1. Para la función tabulada

x	0	1	2	3	4
$f(x)$	1	2	4	8	16

- a) Construya la **tabla de diferencias progresivas** Δy_i .
- b) Determine el polinomio interpolante con Newton hacia adelante.
- c) Calcule $f(2.5)$ y compárelo con $2^{2.5}$.

2. Para los valores:

x	1	2	3	4
$f(x)$	1	8	27	64

- a) Construya la tabla de **diferencias regresivas** ∇y_i .
- b) Aplique la fórmula de Newton hacia atrás para estimar $f(3.4)$.
- c) Compare con el valor exacto de $f(x) = x^3$.

3. Para los datos experimentales:

x	10	20	30	40	50
y	20.5	24.0	32.0	42.5	51.0

- Verifique que los nodos sean equiespaciados.
- Calcule las diferencias progresivas hasta el tercer orden.
- Estime $y(25)$ usando Newton hacia adelante.
- Comente si el uso de un polinomio de orden más alto podría mejorar o empeorar la estabilidad numérica.

Ejercicio 7. Implemente los siguientes ejercicios utilizando las funciones `newton_interp_equispaced()`, `forward_diff_table()` y `backward_diff_table()`.

- Replique los ejemplos manuales anteriores en R y compare los resultados obtenidos manualmente con los valores calculados por el script. Incluya en su reporte:

- La tabla de diferencias generada.
- El valor estimado y el error de truncamiento.
- Una gráfica de los puntos originales y el polinomio interpolante.

- Genere aleatoriamente 5 puntos de una función cuadrática y aplique:

- Interpolación de Lagrange (con la fórmula clásica).
- Interpolación de Newton (diferencias divididas).
- Interpolación hacia adelante (diferencias finitas).

Compare los tres resultados gráficamente y analice diferencias numéricas.

- Usando los datos:

$$x = [0, 1, 2, 3, 4], \quad y = [2.0, 2.7, 4.8, 8.9, 16.0],$$

- Aplique el método automático `newton_interp_equispaced()` para evaluar la función en $x = 1.5, 2.5, 3.5$.
- Reporte el método seleccionado (adelante o atrás) y el error estimado.
- Compare con una función de referencia $f(x) = 2^x$.

Ejercicio 8. Puntos: $(0, 1), (1, 3), (2, 11)$.

- Lagrange / Forma general: Sea $P_2(x) = a_0 + a_1x + a_2x^2$. Con $x = 0 \Rightarrow a_0 = 1$; $x = 1 \Rightarrow 1 + a_1 + a_2 = 3$; $x = 2 \Rightarrow 1 + 2a_1 + 4a_2 = 11$. De $a_1 + a_2 = 2$ y $a_1 + 2a_2 = 5 \Rightarrow a_2 = 3, a_1 = -1$.

$$P_2(x) = 1 - x + 3x^2.$$

- Evaluación: $P_2(1.5) = 1 - 1.5 + 3(2.25) = 6.25$.
- Newton: tabla de diferencias divididas produce los mismos coeficientes.

Ejercicio 9. Puntos: $(1, 2), (2, 5), (3, 10), (4, 17)$.

- Observa que $y = x^2 + 1$ calza exactamente con los datos.

- **Newton:** diferencias

$$\Delta y = \{3, 5, 7\}, \quad \Delta^2 y = \{2, 2\}, \quad \Delta^3 y = \{0\}.$$

- **Polinomio:** de grado 2 (término cúbico nulo), equivale a $P(x) = x^2 + 1$.
- **Evaluación:** $P(2.5) = 2.5^2 + 1 = 7.25$.

Ejercicio 10. Puntos: $(-1, 4), (0, 1), (2, 3)$.

- **Forma general** $P_2(x) = a_0 + a_1x + a_2x^2$. De $x = 0 \Rightarrow a_0 = 1$.
- **Sistema:** $-a_1 + a_2 = 3$ y $a_1 + 2a_2 = 1$. **Solución:** $a_2 = \frac{4}{3}$, $a_1 = -\frac{5}{3}$.

$$P_2(x) = 1 - \frac{5}{3}x + \frac{4}{3}x^2.$$

- **Verifica:** $P_2(-1) = 4$, $P_2(0) = 1$, $P_2(2) = 3$.

Ejercicio 11. Tabla 2^x en $x = 0, 1, 2, 3, 4$: $y = \{1, 2, 4, 8, 16\}$.

- **Diferencias progresivas:** $\Delta y_0 = 1$, $\Delta^2 y_0 = 1$, $\Delta^3 y_0 = 1$, $\Delta^4 y_0 = 1$.
- **Newton adelante (grado 4)**, $x_0 = 0, h = 1, p = 2.5$:

$$\begin{aligned} P(2.5) &= y_0 + p\Delta y_0 + \frac{p(p-1)}{2}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{6}\Delta^3 y_0 + \frac{p(p-1)(p-2)(p-3)}{24}\Delta^4 y_0 \\ &= 1 + 2.5 + 1.875 + 0.3125 - 0.0390625 \\ &= 5.6484375. \end{aligned}$$

- **Comparación:** $2^{2.5} = \sqrt{32} \approx 5.65685425$. **Error** $\approx -8.4168 \times 10^{-3}$ (el grado 4 no puede reproducir una función exponencial).

Ejercicio 12. $x = 1, 2, 3, 4$ con $f(x) = x^3 = \{1, 8, 27, 64\}$; estimar $f(3.4)$ por **atrás**.

- **Regresivas en $x_4 = 4$:** $\nabla y_4 = 37$, $\nabla^2 y_4 = 18$, $\nabla^3 y_4 = 6$.
- $h = 1$, $q = (3.4 - 4) = -0.6$.

$$\begin{aligned} P(3.4) &= y_4 + q\nabla y_4 + \frac{q(q+1)}{2}\nabla^2 y_4 + \frac{q(q+1)(q+2)}{6}\nabla^3 y_4 \\ &= 64 - 22.2 - 2.16 - 0.336 \\ &= 39.304. \end{aligned}$$

- **Exacto:** $3.4^3 = 39.304$. (Para polinomio cúbico con 4 nodos, el método reproduce exactamente.)

Ejercicio 13. Datos experimentales equiespaciados ($h = 10$):

x	10	20	30	40	50
y	20.5	24.0	32.0	42.5	51.0

- **Progresivas en $x_0 = 10$:**

$$\Delta y_0 = 3.5, \quad \Delta^2 y_0 = 4.5, \quad \Delta^3 y_0 = -2.0, \quad \Delta^4 y_0 = -2.5.$$

- **Estimar $y(25)$ (adelante, hasta orden 3):** $p = \frac{25-10}{10} = 1.5$.

$$\begin{aligned} P(25) &= y_0 + p\Delta y_0 + \frac{p(p-1)}{2}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{6}\Delta^3 y_0 \\ &= 20.5 + (1.5)(3.5) + \frac{1.5 \cdot 0.5}{2}(4.5) + \frac{1.5 \cdot 0.5 \cdot (-0.5)}{6}(-2.0) \\ &= 27.5625. \end{aligned}$$

- **Término siguiente (cota pragmática):**

$$\frac{p(p-1)(p-2)(p-3)}{24}\Delta^4 y_0 = \frac{1.5 \cdot 0.5 \cdot (-0.5) \cdot (-1.5)}{24}(-2.5) \approx \boxed{-0.0586}.$$

(Magnitud $\approx 5.86 \times 10^{-2}$: sugiere el orden del error de truncamiento.)

Ejercicio 14. Para 5 puntos de una cuadrática simulada $y = ax^2 + bx + c$ con ruido $N(0, \sigma)$:

- Lagrange (clásico) y Newton (divididas) deben coincidir en precisión con datos sin ruido.
- Con ruido, truncar el orden (p.ej. usar 3-4 nodos o limitar `max_order`) reduce oscilaciones y mejora estabilidad.

Ejercicio 15. Con $x = \{0, 1, 2, 3, 4\}$, $y = \{2.0, 2.7, 4.8, 8.9, 16.0\}$:

- `newton_interp_equispaced` seleccionará adelante cerca de x_0 y atrás cerca de x_4 (o el que quede más cercano).
- La columna `est_trunc_error` reportará el tamaño del término siguiente (guía de error).
- Al comparar con 2^x , se observarán discrepancias crecientes fuera del rango central (interpolación no reproduce exponenciales con grado bajo de forma exacta).

Ejemplo 19. Datos:

$$(x_i, y_i) = (0, 1), (1, 2.2), (2, 2.8), (3, 3.6).$$

Pendientes por tramo

$$b_0 = \frac{2.2 - 1}{1 - 0} = 1.2, \quad b_1 = \frac{2.8 - 2.2}{2 - 1} = 0.6, \quad b_2 = \frac{3.6 - 2.8}{3 - 2} = 0.8.$$

Por tramos:

$$S(x) = \begin{cases} 1 + 1.2(x - 0), & 0 \leq x \leq 1, \\ 2.2 + 0.6(x - 1), & 1 \leq x \leq 2, \\ 2.8 + 0.8(x - 2), & 2 \leq x \leq 3. \end{cases}$$

Evaluaciones:

$$S(1.7) = 2.2 + 0.6(0.7) = 2.62, \quad S(2.4) = 2.8 + 0.8(0.4) = 3.12.$$

Verificaciones:

$$S(1^-) = 2.2, \quad S(1^+) = 2.2; \quad S(2^-) = 2.8, \quad S(2^+) = 2.8.$$

(Sólo continuidad de la función; derivadas cambian en los nudos.)

Ejemplo 20. *Datos:*

$$(x_0, y_0) = (0, 0), \quad (x_1, y_1) = (1, 1), \quad (x_2, y_2) = (2, 0), \quad (x_3, y_3) = (3, 1).$$

Paso $h_i = 1$. Pendientes locales:

$$m_0 = \frac{1-0}{1} = 1, \quad m_1 = \frac{0-1}{1} = -1, \quad m_2 = \frac{1-0}{1} = 1.$$

Spline natural: $M_0 = M_3 = 0$. Sistema para M_1, M_2 (tridiagonal clásico):

$$\begin{cases} M_0 + 4M_1 + M_2 = 6(m_1 - m_0) = 6(-1 - 1) = -12, \\ M_1 + 4M_2 + M_3 = 6(m_2 - m_1) = 6(1 - (-1)) = 12. \end{cases}$$

Con $M_0 = M_3 = 0$:

$$4M_1 + M_2 = -12, \quad M_1 + 4M_2 = 12.$$

Resolviendo:

$$M_2 = 4, \quad M_1 = -4, \quad (M_0 = 0, \quad M_3 = 0).$$

Coefficientes por tramo ($h_i = 1$):

$$\begin{aligned} a_i &= y_i, \\ b_i &= m_i - \frac{(2M_i + M_{i+1})h_i}{6}, \\ c_i &= \frac{M_i}{2}, \\ d_i &= \frac{M_{i+1} - M_i}{6h_i}. \end{aligned}$$

Tramo $[0, 1]$ ($i = 0$): $m_0 = 1$, $M_0 = 0$, $M_1 = -4$:

$$a_0 = 0, \quad b_0 = 1 - \frac{(0-4)}{6} = \frac{5}{3}, \quad c_0 = 0, \quad d_0 = \frac{-4-0}{6} = -\frac{2}{3}.$$

$$S_0(x) = \frac{5}{3}x - \frac{2}{3}x^3.$$

Tramo $[1, 2]$ ($i = 1$): $m_1 = -1$, $M_1 = -4$, $M_2 = 4$:

$$a_1 = 1, \quad b_1 = -1 - \frac{(2(-4)+4)}{6} = -\frac{4}{3}, \quad c_1 = -2, \quad d_1 = \frac{4-(-4)}{6} = \frac{4}{3},$$

$$S_1(x) = 1 - \frac{4}{3}(x-1) - 2(x-1)^2 + \frac{4}{3}(x-1)^3.$$

Tramo $[2, 3]$ ($i = 2$): $m_2 = 1$, $M_2 = 4$, $M_3 = 0$:

$$a_2 = 0, \quad b_2 = 1 - \frac{(8+0)}{6} = -\frac{1}{3}, \quad c_2 = 2, \quad d_2 = \frac{0-4}{6} = -\frac{2}{3},$$

$$S_2(x) = -\frac{1}{3}(x-2) + 2(x-2)^2 - \frac{2}{3}(x-2)^3 + 0 \quad (\text{y desplazar por } y_2 = 0).$$

Evaluaciones:

$$S(0) = 0, \quad S(1) = 1, \quad S(2) = 0, \quad S(3) = 1, \quad S'(0) = \frac{5}{3}, \quad S''(0) = 0, \quad S'''(3) = 0.$$

(Estructura suave: continuidad de S, S', S'' en $x = 1, 2$.)

Ejemplo 21. Datos:

$$(0, 0), \quad (1, 2), \quad (2, 3), \quad S'(0) = 1, \quad S'(2) = 0.$$

$h_0 = h_1 = 1$, $m_0 = 2$, $m_1 = 1$. Frontera clamped y ecuaciones interiores:

$$\begin{cases} 2h_0M_0 + h_0M_1 = 6(m_0 - S'(0)) = 6, \\ h_0M_0 + 2(h_0 + h_1)M_1 + h_1M_2 = 6(m_1 - m_0) = -6, \\ h_1M_1 + 2h_1M_2 = 6(S'(2) - m_1) = -6. \end{cases}$$

Resolución:

$$M_0 = 4, \quad M_1 = -2, \quad M_2 = -2.$$

Coefficientes (con $h = 1$):

$$\begin{aligned} a_0 &= 0, & b_0 &= m_0 - \frac{2M_0+M_1}{6} = 2 - \frac{8-2}{6} = 1, \\ c_0 &= \frac{M_0}{2} = 2, & d_0 &= \frac{M_1-M_0}{6} = -1, \\ a_1 &= 2, & b_1 &= m_1 - \frac{2M_1+M_2}{6} = 1 - \frac{-4-2}{6} = 2, \\ c_1 &= \frac{M_1}{2} = -1, & d_1 &= \frac{M_2-M_1}{6} = 0. \end{aligned}$$

Por tramos:

$$S(x) = \begin{cases} x + 2x^2 - x^3, & 0 \leq x \leq 1, \\ 2 + 2(x-1) - (x-1)^2, & 1 \leq x \leq 2. \end{cases}$$

Verif.: $S(0) = 0$, $S(1) = 2$, $S(2) = 3$; $S'(0) = 1$ y $S'(2) = 0$ (clamped cumplido).

Ejemplo 22. 1. **Spline lineal:** Con $(0, 1.2)$, $(1, 2.0)$, $(3, 3.1)$, $(4, 4.0)$:

a) Escriba $S_i(x)$ por tramos y evalúe $S(2.5)$.

b) Grafique a mano los segmentos y comente la falta de suavidad en los nudos.

2. **Spline cúbico natural:** Con $(0, 0)$, $(1, 1)$, $(2, 0)$, $(3, 1)$:

a) Arme el sistema tridiagonal para M_1, M_2 y resuélvalo.

b) Obtenga los coeficientes (a_i, b_i, c_i, d_i) y evalúe $S(1.5)$ y $S(2.3)$.

3. **Spline cúbico completo:** Con $(0, 0)$, $(1, 2)$, $(2, 3)$ y $S'(0) = 1$, $S'(2) = 0$:

a) Arme el sistema para M_0, M_1, M_2 .

b) Calcule $S(0.5)$ y $S(1.8)$.

1.9. Implementaciones Computacionales

Pseudocódigo (forma clásica)

Sea $x[0..n]$, $y[0..n]$ los datos, y xq el punto a evaluar.

```
ALGORITMO LagrangeEval(x[0..n], y[0..n], xq):
  p := 0
  para i := 0..n hacer
    Li := 1
    para j := 0..n hacer
      si j != i entonces
        Li := Li * (xq - x[j]) / (x[i] - x[j])
    fin si
  fin para
  p := p + y[i] * Li
fin para
retornar p
FIN
```

Pseudocódigo (baricéntrico, estable)

Precompute los pesos baricéntricos $w_i = \frac{1}{\prod_{j \neq i} (x_i - x_j)}$ una sola vez.

```
ALGORITMO BarycentricPrecompute(x[0..n]):
  para i := 0..n hacer
    wi := 1
    para j := 0..n hacer
      si j != i entonces
        wi := wi * 1.0 / (x[i] - x[j])
    fin si
  fin para
  w[i] := wi
fin para
retornar w
FIN

ALGORITMO BarycentricEval(x[0..n], y[0..n], w[0..n], xq):
  // Si xq coincide con un nodo, devuelve su y exacta
  para i := 0..n hacer
    si xq == x[i] entonces retornar y[i]
  fin para
  num := 0 ; den := 0
  para i := 0..n hacer
    term := w[i] / (xq - x[i])
    num := num + term * y[i]
    den := den + term
  fin para
  retornar num / den
FIN
```

Implementación en R (forma clásica y baricéntrica)

```
# --- Forma clasica  $O(n^2)$  por evaluacion ---
lagrange_eval <- function(x, y, xq) {
  # x, y: vectores de longitud n+1
  # xq: escalar o vector
  sapply(xq, function(xx) {
    n <- length(x) - 1
    p <- 0
    for (i in 0:n) {
      Li <- 1
      for (j in 0:n) {
        if (j != i) {
          Li <- Li * (xx - x[j+1]) / (x[i+1] - x[j+1])
        }
      }
      p <- p + y[i+1] * Li
    }
    p
  })
}
```

```
# --- Pesos baricentricos (precomputo  $O(n^2)$ ) ---
barycentric_weights <- function(x) {
  n <- length(x) - 1
  w <- rep(1, n+1)
  for (i in 0:n) {
    for (j in 0:n) {
      if (j != i) w[i+1] <- w[i+1] / (x[i+1] - x[j+1])
    }
  }
  w
}
```

```
# --- Evaluacion baricentrica  $O(n)$  por punto ---
barycentric_eval <- function(x, y, w, xq) {
  sapply(xq, function(xx) {
    # Coincidencia exacta con un nodo
    idx <- which(xx == x)
    if (length(idx) > 0) return(y[idx[1]])
    terms <- w / (xx - x)
    sum(terms * y) / sum(terms)
  })
}
```

```
# --- Ejemplo de uso ---
# Datos:  $f(x)=x^2$  en nodos 0,1,2
x <- c(0,1,2); y <- x^2
```

```

# Evaluar en una malla
xx <- seq(0,2,length.out=21)
yy_lagr <- lagrange_eval(x,y,xx)

w <- barycentric_weights(x)
yy_bar <- barycentric_eval(x,y,w,xx)

# yy_lagr y yy_bar deben coincidir (salvo error redondeo) con xx^2
# plot(xx, yy_lagr, type="l"); points(x,y)

```

Pseudocódigo

```

ALGORITMO NewtonDiffDiv(x[0..n], y[0..n]):
  para i := 0..n hacer
    F[i,0] := y[i]
  fin para
  para j := 1..n hacer
    para i := 0..n-j hacer
      F[i,j] := (F[i+1,j-1] - F[i,j-1]) / (x[i+j] - x[i])
    fin para
  fin para
  retornar F[0,0..n] // Coeficientes f[x0], f[x0,x1], ...

```

FIN

```

ALGORITMO NewtonEval(x[0..n], F[0,0..n], xq):
  p := F[0,0]
  prod := 1
  para j := 1..n hacer
    prod := prod * (xq - x[j-1])
    p := p + F[0,j]*prod
  fin para
  retornar p

```

FIN

Implementación en R

```

# --- Tabla de diferencias divididas ---
newton_diffdiv <- function(x, y) {
  n <- length(x)
  F <- matrix(0, n, n)
  F[,1] <- y
  for (j in 2:n) {
    for (i in 1:(n-j+1)) {
      F[i,j] <- (F[i+1,j-1] - F[i,j-1]) / (x[i+j-1] - x[i])
    }
  }
  F
}

```

```

# --- Evaluacion del polinomio de Newton ---
newton_eval <- function(x, F, xq) {
  n <- length(x)
  sapply(xq, function(xx) {
    p <- F[1,1]
    prod <- 1
    for (j in 2:n) {
      prod <- prod * (xx - x[j-1])
      p <- p + F[1,j]*prod
    }
    p
  })
}

# --- Ejemplo ---
x <- c(0,1,2,3)
y <- c(1,0,-1,2)
F <- newton_diffdiv(x,y)
xx <- seq(0,3,length.out=41)
yy <- newton_eval(x,F,xx)

# plot(xx,yy,type="l",col="blue"); points(x,y,col="red",pch=19)

```

```

ALGORITMO TablaDiferenciasProgresivas(x[0..n], y[0..n]):
  // Requiere nodos equiespaciados:  $x[i] = x[0] + i \cdot h$ 
  VerificarEquiespaciado(x)
  T := matriz (n+1) x (n+1) llena de 0
  para i := 0..n hacer
    T[i,0] := y[i]
  fin para
  para j := 1..n hacer
    para i := 0..n-j hacer
      T[i,j] := T[i+1,j-1] - T[i,j-1] //
    fin para
  fin para
  retornar T
FIN

```

```

ALGORITMO NewtonAdelanteEval(x[0..n], T, xq):
  // T es la tabla de diferencias progresivas
  h := x[1] - x[0]
  p := (xq - x[0]) / h
  P := T[0,0]
  prod := 1
  para k := 1..n hacer

```

```

        prod := prod * (p - (k-1))          // p(p-1)(p-2)...
        P := P + (prod / k!) * T[0,k]       // usa factorial de k
    fin para
    retornar P
FIN

```

```

ALGORITMO TablaDiferenciasRegresivas(x[0..n], y[0..n]):
    VerificarEquiespaciado(x)
    T := matriz (n+1) x (n+1) llena de 0
    para i := 0..n hacer
        T[i,0] := y[i]
    fin para
    para j := 1..n hacer
        para i := n..j hacer
            T[i,j] := T[i,j-1] - T[i-1,j-1] //
        fin para
    fin para
    retornar T
FIN

```

```

ALGORITMO NewtonAtrasEval(x[0..n], T, xq):
    h := x[1] - x[0]
    q := (xq - x[n]) / h
    P := T[n,0]
    prod := 1
    para k := 1..n hacer
        prod := prod * (q + (k-1))          // q(q+1)(q+2)...
        P := P + (prod / k!) * T[n,k]
    fin para
    retornar P
FIN

```

```

# =====
# Utilidades
# =====

is_equispaced <- function(x, tol = 1e-9) {
    if (length(x) < 2) return(TRUE)
    h <- diff(x)
    max(abs(h - h[1])) < tol
}

falling_prod <- function(p, k) {
    # p(p-1)(p-2)...(p-k+1), falling factorial

```

```

    if (k == 0) return(1)
    prod(p - 0:(k-1))
}

rising_prod <- function(q, k) {
  # q(q+1)(q+2)...(q+k-1), rising factorial
  if (k == 0) return(1)
  prod(q + 0:(k-1))
}

# =====
# 1) Tabla de diferencias progresivas (Newton hacia adelante)
# =====
forward_diff_table <- function(x, y, check_equispaced = TRUE) {
  n <- length(x)
  if (length(y) != n) stop("x e y deben tener la misma longitud.")
  if (check_equispaced && !is_equispaced(x)) {
    stop("Los nodos no son equiespaciados.")
  }
  T <- matrix(0, nrow = n, ncol = n)
  T[, 1] <- y
  if (n >= 2) {
    for (j in 2:n) {
      for (i in 1:(n - j + 1)) {
        T[i, j] <- T[i + 1, j - 1] - T[i, j - 1] # Delta^j y_i
      }
    }
  }
  T
}

# =====
# 2) Evaluacion Newton hacia adelante
#    $P(x) = y_0 + p y'_0 + \frac{p(p-1)}{2!} y''_0 + \dots$ 
# =====
newton_forward_eval <- function(x, T, xq) {
  n <- length(x)
  h <- x[2] - x[1]
  p <- (xq - x[1]) / h
  # T[1, k] contiene  $y^{(k-1)}_0$ , con k empezando en 1
  P <- T[1, 1]
  for (k in 2:n) {
    coef <- falling_prod(p, k - 1) / factorial(k - 1)
    P <- P + coef * T[1, k]
  }
  P
}

```

```

# =====
# 3) Tabla de diferencias regresivas (Newton hacia atras)
# =====
backward_diff_table <- function(x, y, check_equispaced = TRUE) {
  n <- length(x)
  if (length(y) != n) stop("x e y deben tener la misma longitud.")
  if (check_equispaced && !is_equispaced(x)) {
    stop("Los nodos no son equiespaciados.")
  }
  T <- matrix(0, nrow = n, ncol = n)
  T[, 1] <- y
  if (n >= 2) {
    for (j in 2:n) {
      for (i in n:j) {
        T[i, j] <- T[i, j - 1] - T[i - 1, j - 1] # Nabla^j y_i
      }
    }
  }
  T
}

# =====
# 4) Evaluacion Newton hacia atras
#    $P(x) = y_n + q y'_n + q(q+1)/2! \cdot y''_n + \dots$ 
# =====
newton_backward_eval <- function(x, T, xq) {
  n <- length(x)
  h <- x[2] - x[1]
  q <- (xq - x[n]) / h
  P <- T[n, 1]
  for (k in 2:n) {
    coef <- rising_prod(q, k - 1) / factorial(k - 1)
    P <- P + coef * T[n, k]
  }
  P
}

# =====
# 5) Helpers para tabla "bonita" (opcional)
# =====
format_diff_table <- function(x, T, type = c("forward", "backward")) {
  type <- match.arg(type)
  n <- length(x)
  df <- data.frame(x = x, y = T[, 1])
  colnames(df) <- c("x", "y")
  for (j in 2:n) {
    colname <- if (type == "forward") paste0("Delta^", j - 1)
    else paste0("Nabla^", j - 1)
  }
}

```

```

    df[[colname]] <- T[, j]
  }
  df
}

# =====
# 6) Ejemplos de uso
# =====

## Ejemplo A:  $f(x) = x^2$ ,  $x = 0,1,2,3$ ; evaluar en 0.5 (adelante)
xA <- 0:3
yA <- xA^2
TA <- forward_diff_table(xA, yA)          # tabla
PA <- newton_forward_eval(xA, TA, 0.5)     # ~ 0.25
dfA <- format_diff_table(xA, TA, "forward")
# print(dfA); cat("P(0.5) =", PA, "\n")

## Ejemplo B:  $f(x) = x^3$ ,  $x = 1,2,3,4$ ; evaluar en 1.5 (adelante)
xB <- 1:4
yB <- xB^3
TB <- forward_diff_table(xB, yB)
PB <- newton_forward_eval(xB, TB, 1.5)     # ~ 3.375
dfB <- format_diff_table(xB, TB, "forward")

## Ejemplo C:  $f(x) = x^2$ ,  $x = 0,1,2,3$ ; evaluar en 2.6 (atras)
xC <- 0:3
yC <- xC^2
TC <- backward_diff_table(xC, yC)
PC <- newton_backward_eval(xC, TC, 2.6)    # ~ 6.76
dfC <- format_diff_table(xC, TC, "backward")

## Ejemplo D:  $f(x) = x^3$ ,  $x = 1,2,3,4$ ; evaluar en 3.2 (atras)
xD <- 1:4
yD <- xD^3
TD <- backward_diff_table(xD, yD)
PD <- newton_backward_eval(xD, TD, 3.2)    # ~ 32.768
dfD <- format_diff_table(xD, TD, "backward")

```